



TD 09 - Terraform

Dans ce TD, vous allez découvrir Terraform, un outil d'**Infrastructure as Code** (IaC) développé par HashiCorp. Terraform permet d'automatiser la création, la gestion et la mise à jour des infrastructures de manière **déclarative**. Grâce à lui, vous pouvez définir votre architecture sous forme de code et la déployer de manière **reproductible** et **cohérente**.

L'objectif de cet exercice est de vous familiariser avec les concepts de base de Terraform en mettant en place une infrastructure simple. Vous apprendrez à écrire un fichier de configuration, à initialiser un projet Terraform et à appliquer vos changements pour **provisionner** des ressources automatiquement.

Objectifs

À l'issue de ce TD, vous serez capable de créer une infrastructure sur Microsoft Azure en utilisant un script Terraform.

PRÉ-REQUIS

Connaissance de base des commandes shell, de Docker et du déploiement sur Microsoft Azure.

Installation

Pour installer Terraform vous pouvez suivre les étapes suivantes :

1. Télécharger le binaire Terraform [en consultant la page officielle](#)
2. Extraire et placer le binaire dans un dossier accessible
3. Ajouter ce dossier à la variable d'environnement `PATH`

Vérifiez l'installation de Terraform en exécutant la commande `terraform version`

Script et commandes

Un script Terraform est un fichier écrit en HCL ([HashiCorp Configuration Language](#)) qui décrit l'infrastructure qu'on souhaite créer, modifier ou supprimer de manière **déclarative**. Un script Terraform est généralement nommés `main.tf` et est décomposé en **Blocs**.

❗ BLOCS

Un Bloc est une structure de configuration qui regroupe des arguments et des valeurs associées. Il est défini par un **type** et contient éventuellement des blocs imbriqués.

Un bloc suit *généralement* cette syntaxe :

```
1 bloc_type "option, nom ou type" "nom_optionnel" {
2   clé = valeur
3   clé2 = valeur2
4
5   bloc_imbriqué {
6     clé = valeur
7   }
8 }
```

Le premier bloc que vous allez rencontrer est le bloc `terraform {}`.

`main.tf`

```
1 terraform {
2   required_version = ">= 1.11.0"
3 }
```

Ce bloc est utilisé pour spécifier des informations liées à l'environnement Terraform global. Dans l'exemple proposé il spécifie la version minimale de Terraform pour exécuter ce script.

Vous pouvez consulter [la documentation du bloc terraform](#) pour connaître les arguments autorisés.

En général un script Terraform est composé des types de blocs suivants :

- `provider` : Définit la connexion à un service (Azure, Docker, AWS...).
- `variable` : Déclare une variable utilisée dans le script Terraform.
- `ressource` : Définit ce qui doit être créé (VM, conteneurs, bases de données,...).

- `output` : Affiche des informations utiles après l'exécution, comme les adresses IP des serveurs créés.

Si le script Terraform devient trop long, il peut être divisé en plusieurs fichiers (`variables.tf`, `outputs.tf`,...). L'ensemble de ces scripts forment le projet Terraform.

Bloc de type provider

Un **provider** dans Terraform est un plugin qui permet d'interagir avec une plateforme. Il agit comme un connecteur entre Terraform et l'infrastructure à gérer. La liste des providers est disponible sur le [registre Terraform](#).

Dans un premier temps, vous allez utiliser le **provider local** pour apprendre les commandes Terraform essentielles avant de passer à la gestion d'infrastructure dans le cloud. Comme son nom l'indique, ce provider permet de créer et de manipuler des ressources sur votre machine locale, ce qui est idéal pour se familiariser avec Terraform sans nécessiter de connexion à un cloud comme Azure.

Dans un nouveau dossier **créez** le fichier `main.tf` suivant :

`main.tf`

```
1 terraform {
2   required_providers {
3     local = {
4       source = "hashicorp/local"
5       version = "2.7.0"
6     }
7   }
8 }
9
10 provider "local" {
11   # Configuration options
12 }
```

Le bloc `terraform{}` est enrichi du sous bloc de type `required_providers`. Ce sous-bloc indique les providers nécessaires pour ce projet. [La configuration de ce bloc est consultable en ligne](#). En résumé ce sous-bloc est composé des arguments :

- `local` : indique le nom du provider.

- `source` : spécifie la source du provider, ici `hashicorp/local`, le provider du registre officiel de HashiCorp.
- `version` : spécifie la version du provider.

Vous constatez également qu'un nouveau bloc de type provider est ajouté : `provider "local"` `{}`. Ce bloc permet de configurer le provider. Dans ce premier exemple aucune configuration n'est requise. Le code exemple pour utiliser ce provider provient directement de l'onglet *USE PROVIDER* du [registre terraform](#)

Afin de télécharger le provider, **exécutez** la commande suivante :

```
terraform init
```

Vérifiez qu'un dossier `.terraform` est créé dans votre dossier de travail. Ce dossier contient le provider local.

❗ DEPENDENCY-LOCK

Le fichier `.terraform.lock.hcl` [dependency-lock](#) créé après l'exécution de la commande `terraform init` verrouille les versions précises des providers utilisés dans le projet Terraform. Ainsi, même si une nouvelle version d'un provider est publiée, Terraform utilisera toujours la même version que celle enregistrée dans ce fichier, garantissant la stabilité du projet.

Si vous souhaitez forcer une mise à jour la commande `terraform init -upgrade` devra être utilisée.

Bloc de type resource

Le bloc de type resource est utilisé pour définir une ressource, c'est à dire un élément que Terraform va créer, modifier ou supprimer.

Modifiez le fichier `main.tf` pour intégrer la gestion d'une ressource comme présenté ci-dessous.

```
main.tf
```

```
1 terraform {  
2   required_providers {
```

```
3     local = {
4         source = "hashicorp/local"
5         version = "2.7.0"
6     }
7 }
8 }
9
10 provider "local" {}
11
12 resource "local_file" "example" {
13     content = "Hello, Terraform!"
14     filename = "${path.module}/example.txt"
15 }
```

La documentation de ce bloc précise qu'une ressource est définie via un type, `local_file` dans notre exemple. Ensuite le nom `example` est associé à cette ressource afin de pouvoir y faire référence dans le script.

Ce bloc est composé de deux arguments de configuration :

- `content` : définit le contenu du fichier.
- `filename` : définit le chemin du fichier créé. Ce chemin est spécifié par `path.module`, qui fait référence au répertoire où se trouve le fichier `.tf` actuel, suivi du nom du fichier `example.txt`.

L'utilisation de cette ressource est expliquée dans la documentation du provider local.

Afin de connaître les actions que Terraform va entreprendre si vous exécutez ce script, exécutez la commande :

```
terraform plan
```

Le résultat de la commande doit vous indiquer les propriétés du fichier `example.txt` qui va être créé.

```
1 Terraform will perform the following actions:
2
3 # local_file.example will be created
4 + resource "local_file" "example" {
5     + content = "Hello, Terraform!"
6     + content_base64sha256 = (known after apply)
7     + content_base64sha512 = (known after apply)
```

```
8      + content_md5          = (known after apply)
9      + content_sha1         = (known after apply)
10     + content_sha256        = (known after apply)
11     + content_sha512         = (known after apply)
12     + directory_permission = "0777"
13     + file_permission        = "0777"
14     + filename               = "./example.txt"
15     + id                     = (known after apply)
16   }
17
18 Plan: 1 to add, 0 to change, 0 to destroy.
```

Pour appliquer les modifications de l'infrastructure proposée, utilisez la commande :

```
terraform apply
```

❗ DO YOU WANT TO PERFORM THESE ACTIONS ?

Lors de l'exécution de la commande `apply`, Terraform vous demande à nouveau de vérifier les changements qui vont être apportés. En effet il se peut que entre l'exécution de la commande `plan` et `apply` des mises à jours ont modifié l'état du système. Vous pouvez éviter cette dernière confirmation via la commande `terraform apply --auto-approve`

Vérifiez que le fichier `example.txt` est créé après l'exécution de la commande.

État courant des ressources

Suite à la commande `terraform apply` un fichier JSON `terraform.tfstate` a été créé. Ce fichier conserve un suivi précis de l'état actuel des ressources, c'est grâce à lui que Terraform peut comparer l'état actuel avec la configuration déclarée dans le script pour déterminer ce qui doit être ajouté, modifié ou supprimé.

Si vous essayez à nouveau la commande `terraform apply`, vous devriez obtenir un résultat similaire à celui-ci :

```
1 local_file.example: Refreshing state...
  [id=42086c02e03bf671ddf621ed9922f52f2c7a605c]
2
3 No changes. Your infrastructure matches the configuration.
4
```

- 5 Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.
- 6
- 7 Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

EXERCICE A : SUPPRESSION DE L'ÉTAT

Supprimez le fichier `terraform.tfstate` et exécutez la commande `terraform apply`.
Que se passe-t-il ?

Pour obtenir la liste des ressources actuelles, exécutez la commande

```
terraform state list
```

Pour obtenir des détails sur une ressource particulière, la commande `terraform state show` peut vous aider. Vous devez simplement spécifier le nom de votre ressource. Par exemple le fichier a été créé via `resource "local_file" "example"`, son nom est `local_file.example`.

Exécutez la commande ci-dessous pour obtenir les informations sur ce fichier.

```
terraform state show local_file.example
```

Destruction des ressources

Si vous souhaitez supprimer toutes les ressources définies dans la configuration Terraform, utilisez la commande suivante :

```
terraform destroy
```

DESTRUCTION DES RESSOURCES

- cette action est irréversible
- Terraform ne peut supprimer que ce qu'il gère, si des ressources ont été modifiées manuellement, Terraform pourrait ne pas les supprimer correctement.

Variables

Déclarer une variable

Les variables peuvent être définies dans le fichier `main.tf`. Pour structurer son projet Terraform il est d'usage de les créer dans un fichier séparé.

La syntaxe de base pour **déclarer une variable** est la suivante :

```
1 variable "nom_de_la_variable" {
2   type      = string
3   description = "Description de la variable"
4   default   = "valeur_par_défaut"
5 }
```

La liste des arguments du bloc `variable` et leurs valeurs est [consultable en ligne](#).

EXERCICE B : SECRET

Consultez la documentation de Terraform pour déterminer quel argument faut-il ajouter au bloc de type variable pour que sa valeur ne soit jamais affichée dans le terminal ?

Dans le dossier de votre projet Terraform, **créez** le fichier `variables.tf` avec le contenu suivant :

variables.tf

```
1 variable "file_content" {
2   description = "Contenu du fichier"
3   type      = string
4   default   = "Hello, Terraform with variables!"
5 }
6
7 variable "file_name" {
8   description = "Nom du fichier"
9   type      = string
10  default    = "my_file.txt"
11 }
```

Pour utiliser la variable dans le script `main.tf`, il suffit d'y faire référence via le mot clé `var`. Par exemple `var.file_content` fait référence à la variable `file_content`.

Modifiez le fichier `main.tf` comme ci-dessous.

main.tf

```
1 terraform {
2   required_providers {
3     local = {
4       source = "hashicorp/local"
5       version = "2.7.0"
6     }
7   }
8 }
9
10 provider "local" {}
11
12 resource "local_file" "example" {
13   content = var.file_content
14   filename = "${path.module}/${var.file_name}"
15 }
```

Exécutez le script pour créer le fichier et vérifiez le résultat.



VALIDER UN SCRIPT

La commande `terraform validate` permet de vérifier la syntaxe et la cohérence des fichiers Terraform sans appliquer de modifications à l'infrastructure. Essayez cette commande avant d'appliquer une mise à jour.

Assigner une valeur à une variable

Seul la valeur par défaut d'une variable est utilisable pour l'instant. Si vous souhaitez assigner une valeur à une variable, trois techniques s'offrent à vous :

1. Créer un fichier `*.tfvars` avec les valeurs à assigner.
2. Passer les valeurs en arguments du script.
3. Exporter des variables d'environnements.

Fichier tfvars

Créez dans le repertoire de travail le fichier `terraform.tfvars` ci-dessous.

terraform.tfvars

```
1 file_content="contenu provenant du fichier tfvars"
2 file_name="nouveau_fichier.txt"
```

Exécutez le script pour créer le fichier et vérifiez le nom et le contenu du fichier.

Paramètres de commande

Vous pouvez également passer des variables lors de l'exécution via le paramètre `-var`.

EXERCICE C : PASSER UN PARAMÈTRE

Exécutez le script avec la commande ci-dessous pour créer le fichier et **vérifiez** le nom et le contenu du fichier.

```
terraform apply -var="file_name=file-param.txt"
```

Variables d'environnement

Les variables d'environnements précédées du préfixe `TF_VAR` sont interprétées par Terraform comme des variables de votre projet.

EXERCICE D : PASSER UN PARAMÈTRE

Effacez la valeur de la variable `file_content` du fichier `terraform.tfvars`. **Exportez** ensuite la variable d'environnement `TF_VAR_file_content` avec la valeur "contenu de la variable d'environnement". **Exécutez** le script Terraform pour créer le fichier et **vérifiez** le nom et le contenu du fichier.

Bloc de type output : affichage des résultats

En Terraform, le bloc de type output est utilisé pour afficher des valeurs après l'exécution de la commande `terraform apply`. Il permet d'extraire des informations importantes sur les ressources créées et de les rendre accessibles à l'utilisateur ou à d'autres scripts.

Ce bloc peut être défini dans le fichier `main.tf` mais pour structurer un projet Terraform il est d'usage de les créer dans un fichier séparé : `outputs.tf`.

Créez dans votre dossier de travail le fichier `outputs.tf` présenté ci-dessous.

outputs.tf

```
1 output "file_path" {
2   description = "Chemin du fichier créé"
3   value       = local_file.example.filename
4 }
```

Lors de l'exécution de la commande `terraform apply`, les informations suivantes seront affichées dans le terminal :

Outputs:

```
file_path = "./my_file.txt"
```

AFFICHAGE DES OUTPUTS

Vous pouvez afficher ce résultat par la suite via la commande suivante :

```
terraform output file_path
```

Cette commande s'avère utile pour récupérer les adresses IP des serveurs créés.

Embranchements

Terraform permet d'utiliser des boucles et des conditions pour rendre les configurations plus dynamiques et flexibles.

Créez le projet Terraform composé des fichiers `variables.tf`, `main.tf` et `outputs.tf` ci-dessous.

variables.tf

```
1 variable "file_count" {
2   type    = number
3   default = 3
4 }
```

```
5
6 variable "files_map" {
7     type = map(string)
8     default = {
9         "fileA" = "Contenu du fichier A"
10        "fileB" = "Contenu du fichier B"
11        "fileC" = "Contenu du fichier C"
12    }
13 }
14
15 variable "create_optional_file" {
16     type      = bool
17     default = false
18 }
```

main.tf

```
1 terraform {
2     required_providers {
3         local = {
4             source = "hashicorp/local"
5             version = "2.7.0"
6         }
7     }
8 }
9
10 provider "local" {}
11
12 # Utilisation de count pour créer plusieurs fichiers
13 resource "local_file" "count_example" {
14     count = var.file_count
15
16     filename = "file_${count.index}.txt"
17     content  = "Fichier numéro ${count.index}"
18 }
19
20 # Utilisation de for_each pour créer des fichiers avec des noms dynamiques
21 resource "local_file" "foreach_example" {
22     for_each = var.files_map
23
24     filename = "${each.key}.txt"
25     content  = each.value
26 }
27
28 # Utilisation d'une condition pour créer un fichier optionnel
```

```
29 resource "local_file" "conditional_file" {
30     count = var.create_optional_file ? 1 : 0
31
32     filename = "optional_file.txt"
33     content  = "Ce fichier est créé uniquement si create_optional_file est
    vrai"
34 }
```

output.tf

```
1  # Outputs pour voir les fichiers créés
2  output "count_files" {
3      value = [for f in local_file.count_example : f.filename]
4  }
5
6  output "foreach_files" {
7      value = [for f in local_file.foreach_example : f.filename]
8  }
9
10 output "optional_file" {
11     value = var.create_optional_file ? "optional_file.txt" : "Pas de fichier
    optionnel"
12 }
```

INTERPRÉTER UN SCRIPT

Après avoir **exécuté** la commande `terraform apply`, **expliquez** le fonctionnement des mots clés `count` et `for` et de l'opérateur `?`. Consultez la [documentation sur les expressions terraform](#) pour plus d'informations.

Provider Azure

Dans cette section, vous allez créer une infrastructure sur Microsoft Azure pour déployer l'application `demo-no-db` à partir de son image Docker. Commencez par préparer le code de l'application ainsi que le **Dockerfile** correspondant.

Client Azure

Azure CLI, pour Azure Command-Line Interface, est un outil en ligne de commande permettant de gérer et d'automatiser les ressources et services dans Microsoft Azure. Il permet d'interagir

avec Azure en utilisant des commandes simples plutôt que de passer par le portail web d'Azure.

Installez Azure CLI en suivant [la documentation sur Microsoft Learn](#). **Vérifiez** l'installation via la commande `az version`.

Authentification auprès du fournisseur

Plusieurs méthodes permettent de s'authentifier auprès d'Azure :

- **Azure CLI** (`az login`) : Authentification interactive via un compte utilisateur. Cette commande ouvre un navigateur pour valider la connexion.
- **Service Principal** : Représente une identité d'application dans Azure. Ce service utilise un client ID et un mot de passe (ou certificat). C'est la méthode recommandée pour les scénarios d'automatisation, comme avec Terraform ou GitLab CI.
- **Managed Identity** : Identité gérée automatiquement par Azure, associée à une ressource (ex. : VM, App Service). Elle est idéale lorsque l'exécution de Terraform se fait depuis Azure lui-même.

Dans le cadre de l'automatisation, l'approche recommandée est l'usage d'un Service Principal pour se connecter à Azure.

Connexion via Azure CLI

Avant de créer un Service Principal, vous devez d'abord vous connecter à Azure en local à l'aide de la commande :

```
az login
```

Cette commande ouvre automatiquement une fenêtre de navigateur pour vous permettre de vous authentifier avec vos identifiants Azure.

Une fois connecté, votre terminal affichera les informations liées à votre compte et à votre abonnement, comme ci-dessous :

No	Subscription name	Subscription ID	Tenant
[1] *	Azure for Students Bruxelles Brabant (HE2B)	*****_****_****_****_*****	Haute Ecole

Notez bien votre **Subscription ID**, vous en aurez besoin dans les étapes suivantes pour attribuer les droits adéquats à votre Service Principal.

Si vous souhaitez afficher les informations associées à votre compte, utilisez la commande :

```
az account show
```

Création d'un Service principal

Suivez le tutoriel de [création du Service Principal disponible sur Microsoft Learn](#).

⚠ EXPORT DES VARIABLES ARM

Comme mentionné dans le tutoriel, n'oubliez pas d'exporter les variables d'environnements permettant la connexion à Microsoft Azure via un Service principal. Le nom de ces variables est défini par le provider Azure de Terraform.

```
1 ARM_CLIENT_ID="your-client-id"
2 ARM_CLIENT_SECRET="your-client-secret"
3 ARM_SUBSCRIPTION_ID="your-subscription-id"
4 ARM_TENANT_ID="your-tenant-id"
```

Service principal devient propriétaire des ressources

La première étape consiste à accorder au Service Principal les autorisations nécessaires pour créer les ressources requises.

```
az role assignment create --assignee ${ARM_CLIENT_ID} --role "Owner" --scope /subscriptions/${ARM_SUBSCRIPTION_ID}
```

Le service principal a le rôle **Owner** sur l'ensemble de l'**abonnement**. Il peut créer toutes les ressources nécessaires.

Création du projet Terraform

Créez dans un nouveau dossier les fichiers Terraform suivants :

```
|— main.tf
|— outputs.tf
|— providers.tf
└— variables.tf
```

Commencez par compléter le fichier `providers.tf` contenant la configuration nécessaire pour que Terraform se connecte à Microsoft Azure.

providers.tf

```
1 terraform {
2   required_providers {
3     azurerm = {
4       source = "hashicorp/azurerm"
5       version = "4.66.0"
6     }
7   }
8 }
9
10 provider "azurerm" {
11   features {}
12 }
```

Lors du déploiement d'une première application sur Azure, vous avez utilisé plusieurs types de ressources. Terraform permet d'automatiser la création de ces ressources. La documentation officielle de ces ressources est disponible sur le [le registre terraform](#). Voici la configuration minimale requise pour chacune d'entre elles :

- **azurerm_resource_group** : Crée un groupe de ressources Azure pour regrouper et gérer les ressources liées entre elles.

```
1 resource "azurerm_resource_group" "rg" {
2   name      = var.resource_group_name
3   location = var.location
4 }
```

- **azurerm_user_assigned_identity** : Crée une identité managée pouvant être assignée à une ou plusieurs ressources pour l'authentification.


```
resource "azurerm_user_assigned_identity" "identity" {
  name                       = var.identity_name
  resource_group_name       = azurerm_resource_group.rg.name
  location                  = azurerm_resource_group.rg.location
}
```

- **azurerm_container_registry** : Crée un registre de conteneurs Azure (ACR) pour stocker et gérer des images Docker.

```
resource "azurerm_container_registry" "acr" {
  name                       = var.acr_name
  resource_group_name       = azurerm_resource_group.rg.name
  location                  = azurerm_resource_group.rg.location
  sku                       = "Basic"
  admin_enabled             = true
}
```

- **azurerm_role_assignment** : Attribue un rôle spécifique (comme AcrPull) à une identité sur une ressource pour lui accorder des permissions précises.

```
resource "azurerm_role_assignment" "acr_pull" {
  scope                     = azurerm_container_registry.acr.id
  role_definition_name     = "AcrPull"
  principal_id             = azurerm_user_assigned_identity.identity.principal_id
}
```

- **azurerm_service_plan** : Définit un plan App Service qui spécifie les ressources (CPU/RAM/prix) utilisées par les applications web.

```
resource "azurerm_service_plan" "app_service_plan" {
  name                       = var.service_plan_name
  location                  = var.location
  resource_group_name       = azurerm_resource_group.rg.name
  os_type                   = "Linux"
  sku_name                  = "B1"
}
```

- **azurerm_linux_web_app** : Crée une application web Azure basée sur Linux, pouvant héberger une image Docker personnalisée.

```
resource "azurerm_linux_web_app" "app" {  
  name                = var.web_app_name  
  location             = var.location  
  resource_group_name = azurerm_resource_group.rg.name  
  service_plan_id     = azurerm_service_plan.app_service_plan.id  
}
```

Vous allez utiliser ces configurations dans la suite de l'exercice.

Définition des variables

Ajoutez au fichier `variables.tf` les différentes variables nécessaires au déploiement. Les valeurs par défaut et la description doit vous permettre d'en comprendre le sens.

variables.tf

```
1 variable "location" {  
2   description = "Azure region"  
3   type       = string  
4   default    = "West Europe"  
5 }  
6  
7 variable "resource_group_name" {  
8   description = "Name of the resource group"  
9   type       = string  
10  default    = "rg-container-app"  
11 }  
12  
13 variable "acr_name" {  
14   description = "Name of the Azure Container Registry"  
15   type       = string  
16   default    = "myacrregistry1234"  
17 }  
18  
19 variable "identity_name" {  
20   description = "Name of the user-assigned managed identity"  
21   type       = string  
22   default    = "container-app-identity"  
23 }  
24  
25 variable "service_plan_name" {  
26   description = "Name of the App Service plan"  
27   type       = string  
28   default    = "appserviceplan"
```

```
29 }
30
31 variable "web_app_name" {
32     description = "Name of the Web App"
33     type        = string
34     default     = "my-container-app"
35 }
36 variable "docker_image_name" {
37     description = "Docker image name and tag"
38     type        = string
39     default     = "myapp:latest"
40 }
41
42 variable "web_app_port" {
43     description = "Port on which the web app listens"
44     type        = string
45     default     = "8080"
46 }
```

Vous pourrez adapter lors de l'exécution du script les valeurs associées au nom de l'image et au numéro de port utiliser par l'application.

Définition des ressources

Complétez le fichier `main.tf` avec les ressources à créer dans le cloud. Prenez attention à la ressource définissant l'application web, elle contient la configuration spécifique de votre application.

main.tf

```
1 resource "azurerm_resource_group" "rg" {
2     name      = var.resource_group_name
3     location = var.location
4 }
5
6 resource "azurerm_user_assigned_identity" "identity" {
7     name                        = var.identity_name
8     resource_group_name       = azurerm_resource_group.rg.name
9     location                   = azurerm_resource_group.rg.location
10 }
11
12 resource "azurerm_container_registry" "acr" {
13     name                = var.acr_name
14     resource_group_name = azurerm_resource_group.rg.name
15     location             = azurerm_resource_group.rg.location
```

```
16     sku                                = "Basic"
17     admin_enabled                      = true
18 }
19
20 resource "azurerm_role_assignment" "acr_pull" {
21     scope                                = azurerm_container_registry.acr.id
22     role_definition_name = "AcrPull"
23     principal_id                    =
24     azurerm_user_assigned_identity.identity.principal_id
25 }
26
27 resource "azurerm_service_plan" "app_service_plan" {
28     name                                = var.service_plan_name
29     location                            = var.location
30     resource_group_name = azurerm_resource_group.rg.name
31     os_type                            = "Linux"
32     sku_name                            = "B1"
33 }
34
35 resource "azurerm_linux_web_app" "app" {
36     name                                = var.web_app_name
37     location                            = var.location
38     resource_group_name = azurerm_resource_group.rg.name
39     service_plan_id      = azurerm_service_plan.app_service_plan.id
40     site_config {
41         container_registry_use_managed_identity = true
42         container_registry_managed_identity_client_id =
43         azurerm_user_assigned_identity.identity.client_id
44     }
45     application_stack {
46         docker_image_name = var.docker_image_name
47         docker_registry_url =
48         "https://${azurerm_container_registry.acr.login_server}"
49     }
50     identity {
51         type = "UserAssigned"
52         identity_ids = [azurerm_user_assigned_identity.identity.id]
53     }
54     app_settings = {
55         WEBSITES_PORT = var.web_app_port
56     }
57 }
```

Les détails des arguments utilisés sont consultables via le [registre terraform](#).

Définition des outputs

Afin d'obtenir les informations de connexion à votre application, complétez le fichier `outputs.tf` comme ci-dessous.

outputs.tf

```
1 output "container_registry_login_server" {
2   description = "Adresse du registre ACR"
3   value       = azurerm_container_registry.acr.login_server
4 }
5
6 output "web_app_url" {
7   description = "URL complète de l'application web"
8   value       =
9     "https://${azurerm_linux_web_app.app.default_hostname}/config"
```

Création de l'infrastructure

Les scripts Terraform en placent, vous pouvez créer l'infrastructure via les commandes :

- `terraform init`
- `terraform validate`
- `terraform apply`

Il vous reste une dernière étape, le déploiement de l'image de `demo-no-db` sur le registre de conteneurs.

Déploiement de l'image sur le registre de conteneurs

Sur votre machine, placez-vous dans le dossier où se trouve le **Dockerfile** de l'application `demo-no-db`. Les commandes ci-dessous vont déployer votre image sur le registre de conteneurs.

```
az acr login --name myacrregistry1234
docker build -t myacrregistry1234.azurecr.io/myapp:latest .
docker push myacrregistry1234.azurecr.io/myapp:latest
```



VARIABLES ACR_NAME ET DOCKER_IMAGE_NAME

Si vous avez adapté le nom de votre registre de conteneurs ou le nom de votre image, n'oubliez pas de modifier la valeur `myacrregistry1234` ou la valeur `myapp`.

Si tout est ordre vous devriez pouvoir consulter l'application via l'url fournie par les outputs de Terraform. En cas d'erreur consultez les logs de votre application.

Intégration dans un Pipeline

Pour intégrer la création de l'infrastructure dans le pipeline `gitlab-ci.yml`, il vous suffit de :

- adapter le fichier `.gitignore` pour y inclure les fichiers liés à Terraform,
- ajouter le dossier contenant votre projet Terraform dans le dépôt,
- créer un nouveau job dans le pipeline qui exécute les commandes Terraform.

Cette étape sera réalisée lors des scénarios de synthèse qui récapitulent l'ensemble des étapes vues dans les TDs.